SYSTEM PROGRESS REPORT

[SYSTEM TITLE]

Proponents	**[NAMES]**
Adviser	**[ADVISER NAME]**
School / Program	**[SCHOOL, PROGRAM]**
Report date	**[DATE]**

Status as of 26 October 2026, based on the source code in the project repository (branch *main*, commit 7b2f470).

1. System Overview

The system is a web-based self-service ordering kiosk with integrated real-time stock inventory and transaction management for Zak's Sizzling Hub, a sizzling-plate restaurant. Customers order on an in-store touchscreen kiosk (a Raspberry Pi with a 1024 × 600 display) or on their own phones by scanning a QR code. They browse the menu by category, search or filter by price, add dishes with special instructions, choose dine in or take out, and pay at the counter (online payment through QR Ph is implemented but not yet live). Each order receives a unique order number and an estimated preparation time, and on the in-store kiosk the receipt is also printed on a thermal printer.

Staff use a back office on a separate address to confirm counter payments, follow orders on a kitchen display that updates in real time, adjust stock with a recorded reason, and look up products by barcode. When an order is paid the system deducts stock automatically, either as units or as raw materials according to each dish's recipe, and every open kiosk updates its sold-out items without refreshing. The Administrator additionally manages products, prices, costs and photos, imports products in bulk from CSV or Excel files, views sales analytics with CSV, Excel and PDF export, manages user accounts and roles, and prints the kiosk QR code.

A background worker checks stock levels every hour, sends low-stock e-mail alerts when e-mail is configured, and backs up the database every night to off-site cloud storage. The system is deployed on Railway as separate customer and staff web addresses, an API service, a worker and a managed PostgreSQL database. This report describes only what is present in the source code; every status in Section 4 cites the files that implement it.

Users and roles

Role	What the role can do in the current code
Customer (no sign-in)	Order at the kiosk or on a phone: browse, search, filter by price, add dishes with instructions, choose dine in or take out, pay at the counter, and see (and on the kiosk, print) the receipt.
Staff	Dashboard without cost figures, inventory and raw-material adjustments, barcode lookup, kitchen display, and confirming or cancelling counter orders.
Administrator	Everything Staff can do, plus product management (prices, costs, photos, recipes), bulk import, raw-material set-up, analytics and exports, user and role management, and the kiosk QR code.

Access is enforced by the API on every route (`apps/api/src/middleware/authorize.ts`); the web pages apply the same rules so that users only see what they may use. Figure 1 on the next page shows how the users and the parts of the system work together.

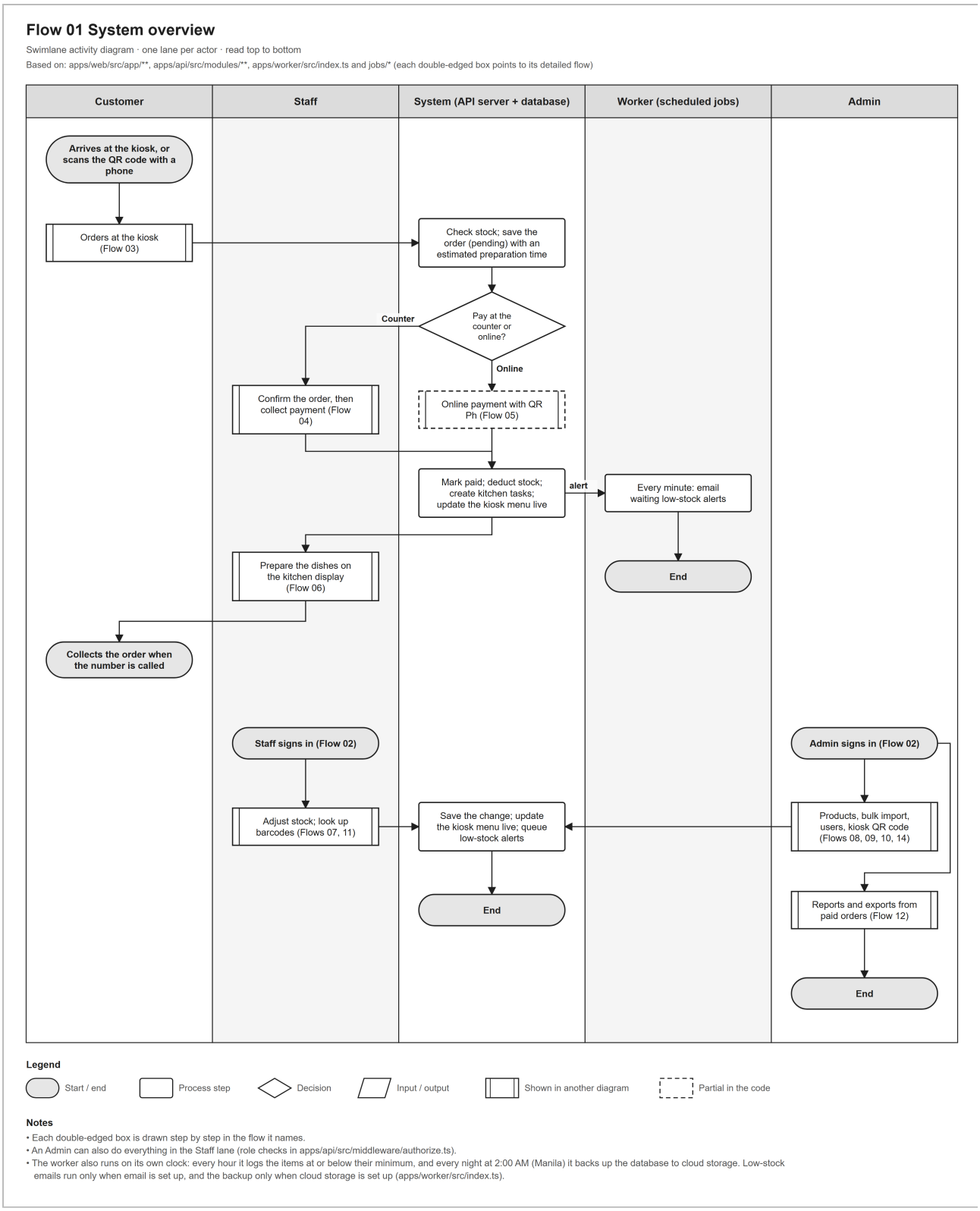


Figure 1. System overview: swimlane activity diagram of the main processes.

2. Technology Stack

Layer	Technology	Use in the system
Language	TypeScript	All application code; one repository with npm workspaces.
Web application	Next.js 16, React 19, Tailwind CSS 4	Customer kiosk, back office and kitchen display (`apps/web`).
API server	Node.js, Express 4, Zod	REST API with validation and role checks (`apps/api`).
Real-time updates	Socket.IO	Stock, new-order and kitchen status events.
Database	PostgreSQL 18, Prisma 6	Schema, migrations and queries (`apps/api/prisma`).
Authentication	JSON Web Tokens, bcrypt	Staff sign-in, anonymous kiosk sessions, password hashing.
Background jobs	node-cron, Nodemailer / Resend, pg_dump	Low-stock checks and e-mails, nightly backups (`apps/worker`).
File handling	ExcelJS, PDFKit, qrcode	Excel import and export, PDF reports, kiosk QR code.
Payments	PayMongo	QR Ph online payment, scanned with any bank or e-wallet app (implemented, not yet live).
Storage	Cloudflare R2	Off-site database backups.
Hosting	Railway	Customer and staff web addresses, API, worker, PostgreSQL.
Kiosk hardware	Raspberry Pi, 1024 × 600 touchscreen, RP21UB-Y 58 mm thermal printer	In-store kiosk; receipts printed through a print service on the Pi.
Testing	Vitest, Supertest	Automated unit, integration and security tests (`apps/api/test`).

3. Overall Progress Summary

The features assessed are those described in the manuscript, grouped into sixteen modules as in the progress report of 25 September 2026, together with two planned activities identified by the proponents (testing PayMongo with real keys and structured testing on the physical kiosk). Four offline and Progressive Web App features were removed from the manuscript by the adviser's decision and are listed but not scored.

Status	Features	Meaning
Completed	63	Works end to end in the code (interface, logic and data saving where relevant).
Partial	11	Started; the missing parts are stated.
Not yet started	5	No implementation in the code.
Removed from scope	4	Removed from the manuscript by adviser decision; not scored.
Total	**83**	**79 scored features**

No feature was found to be a placeholder, that is, a screen or route without real logic behind it.

Overall progress: 86.7%. Each scored feature counts 1 point if Completed, ½ point if Partial and 0 if Not yet started. Overall progress = (63 × 1 + 11 × ½) ÷ 79 = 68.5 ÷ 79 = 86.7%. Features removed from scope are excluded from both the points and the total. Every feature has the same weight, so the figure shows how much of the described functionality exists, not how much effort remains.

Module	Completed	Partial	Not yet started	Removed	Progress
1. Kiosk ordering interface	8	2	0	0	90%
2. Checkout and payments	4	2	2	0	62%
3. Front-desk order handling	3	2	0	0	80%
4. Kitchen display system	4	0	0	0	100%
5. Real-time inventory synchronisation	3	0	0	0	100%
6. Product and catalogue management	5	1	1	0	79%
7. Inventory monitoring and stock adjustments	6	0	0	0	100%
8. Analytics, reports and data export	6	1	0	0	93%
9. Authentication, roles and user management	5	0	0	0	100%
10. Security measures	7	0	0	0	100%
11. Background worker and backups	3	0	0	1	100%
12. Offline support (PWA)	0	0	0	3	—
13. Hardware integration	1	1	0	0	75%
14. QR code and multi-device access	2	0	0	0	100%
15. Deployment	3	0	0	0	100%
16. Testing and quality assurance	3	2	2	0	57%

Module progress uses the same scoring as the overall figure. The progress report of 25 September 2026 averaged the sixteen module percentages and included the offline features; the two figures are therefore not directly comparable.

4. Module and Feature Status

Every feature is listed with its status, a short remark, and the main files that implement it (paths relative to the repository root). A feature is marked Completed only when the interface, the logic and, where relevant, the saving of data are all present in these files.

Feature	Status	Remarks	Main files
1. Kiosk ordering interface			
Product catalogue with names, prices and availability	**Completed**	One category at a time as a grid of tiles; sold-out dishes are dimmed and listed last.	`apps/web/src/components/kiosk/MenuGrid.tsx` `apps/web/src/components/kiosk/ItemTile.tsx` `apps/api/src/modules/products/routes.ts`
Live availability updates	**Completed**	Sold-out status changes on every open kiosk without a page refresh (inventory:updated event).	`apps/web/src/hooks/useCatalog.ts` `apps/api/src/modules/inventory/fulfillment.ts`
Item details: description, ingredients, allergens, nutrition	**Partial**	Displayed on the kiosk. Ingredients, allergens and nutrition have no input screen; they can only be set in the database.	`apps/web/src/components/kiosk/ProductModal.tsx` `apps/web/src/components/kiosk/NutritionGrid.tsx`
Category navigation	**Completed**	Category rail with a picture tile per category.	`apps/web/src/components/kiosk/CategoryRail.tsx`
Keyword search	**Completed**	Searches dish names across all categories.	`apps/web/src/components/kiosk/TopBar.tsx` `apps/web/src/app/page.tsx`
Order (cart): add, change quantity, remove, running total	**Completed**	Saved on the device so a page refresh keeps the order.	`apps/web/src/hooks/useCart.tsx` `apps/web/src/components/kiosk/OrderBar.tsx` `apps/web/src/components/kiosk/CartDrawer.tsx`
Special instructions sent to the kitchen	**Completed**	Category-specific quick options plus a typed note, limited to 140 characters.	`apps/web/src/components/kiosk/ProductModal.tsx` `apps/web/src/lib/instructionChips.ts` `apps/web/src/app/kitchen/page.tsx`
Kiosk session and idle-timeout reset	**Completed**	Warning after 60 s without activity; the order is cleared 20 s later.	`apps/web/src/hooks/useIdleTimer.ts` `apps/web/src/components/kiosk/IdleTimeoutOverlay.tsx`
Product images	**Completed**	Uploaded by the Administrator, reduced in size in the browser, stored in the database.	`apps/web/src/components/admin/ProductForm.tsx` `apps/web/src/lib/imageResize.ts` `apps/api/src/modules/products/image.ts`
Filter by price range or popularity	**Partial**	Price-range filter implemented; no popularity filter.	`apps/web/src/components/kiosk/MenuGrid.tsx` `apps/web/src/lib/priceRanges.ts`

Feature	Status	Remarks	Main files
2. Checkout and payments			
Payment method selection (counter, QR Ph)	**Completed**	QR Ph is shown as unavailable until online payment is configured.	`apps/web/src/app/checkout/page.tsx` `apps/api/src/modules/payments/routes.ts`
Pay-at-counter order placement	**Completed**	The server re-checks availability and stock before accepting the order.	`apps/web/src/app/checkout/page.tsx` `apps/api/src/modules/orders/service.ts`
Online payment through PayMongo (QR Ph)	**Partial**	QR Ph is the one online method; it replaced GCash and Maya on 26 October 2026. Checkout session, redirect and signed webhook are implemented and covered by automated tests with a simulated gateway. Not yet run against PayMongo; production has no keys yet.	`apps/api/src/modules/payments/service.ts` `apps/api/src/lib/paymongo.ts` `apps/web/src/app/checkout/success/page.tsx`
Digital receipt with a unique order number	**Completed**	Order number, order type, items, total, payment method and estimated preparation time.	`apps/web/src/components/kiosk/ReceiptModal.tsx` `apps/api/src/modules/orders/service.ts`
Estimated preparation time	**Completed**	Longest dish + 1 min per extra serving + 2 min per order already in the kitchen; shown at checkout and on the receipt.	`apps/api/src/modules/orders/prepTime.ts` `apps/web/src/app/checkout/page.tsx`
Receipt printing	**Partial**	After a counter order the kiosk sends the receipt to a print service on the Raspberry Pi (58 mm thermal printer). The print service is not in the repository and has not yet been verified with the deployed system.	`apps/web/src/lib/receiptPrinter.ts` `apps/web/src/app/checkout/page.tsx`
QR-code payment option (manuscript wireframe)	**Not yet started**	Not present in the code.	—
Testing of PayMongo with sandbox and live keys	**Not yet started**	Planned. Requires PayMongo keys and the client's approval.	—
3. Front-desk order handling			
List of orders awaiting payment	**Completed**	Unpaid orders and paid orders flagged with a stock issue.	`apps/web/src/app/staff/orders/page.tsx` `apps/api/src/modules/orders/service.ts`
Confirm counter payment	**Completed**	Marks the order paid, deducts stock and sends it to the kitchen in one transaction; a stock shortage rolls everything back.	`apps/api/src/modules/payments/service.ts`
Look up an order by its number	**Completed**	Typed search field (not a keypad).	`apps/web/src/app/staff/orders/page.tsx`

Feature	Status	Remarks	Main files
Transaction history screen	Partial	The API lists recent orders (GET /api/orders); no screen displays it.	apps/api/src/modules/orders/routes.ts
Order status progression and cancellation	Partial	Cancelling an unpaid order is implemented. The statuses preparing, ready and completed are never set, and the customer is not notified when an order is ready.	apps/api/src/modules/orders/service.ts apps/api/src/modules/kitchen/service.ts
4. Kitchen display system			
Real-time order queue (number, time, items, instructions)	Completed	Oldest first; each dish has its own status.	apps/web/src/app/kitchen/page.tsx apps/api/src/modules/kitchen/service.ts
WebSocket push without polling	Completed	order:created and kitchen:task_updated events.	apps/api/src/websocket/index.ts apps/web/src/lib/websocket.ts
Status updates with start and completion times	Completed	Pending, In progress, Completed per dish.	apps/api/src/modules/kitchen/service.ts
Dine-in / take-out indicator	Completed	Chosen by the customer after the welcome screen; take-out orders are highlighted.	apps/web/src/app/kitchen/page.tsx apps/web/src/components/kiosk/OrderTypeScreen.tsx
5. Real-time inventory synchronisation			
Automatic stock deduction when an order is paid	Completed	Atomic deduction; stock never goes below zero.	apps/api/src/modules/inventory/fulfillment.ts
Live stock broadcast to kiosks and back office	Completed	inventory:updated event after every change.	apps/api/src/modules/inventory/fulfillment.ts apps/api/src/modules/inventory/service.ts
Raw-material (ingredient) deduction by recipe	Completed	Dishes can use a recipe; servings available are computed from raw-material stock.	apps/api/src/modules/raw-materials/service.ts apps/api/src/modules/inventory/fulfillment.ts apps/web/src/app/admin/raw-materials/page.tsx
6. Product and catalogue management			
Create, edit and delete products	Completed	Field validation; deletion is refused for products with order history.	apps/web/src/app/admin/products/page.tsx apps/web/src/components/admin/ProductForm.tsx apps/api/src/modules/products/service.ts
Bulk import from CSV, matched by barcode	Completed	Every row is checked first (dry run); import is enabled only when no row has an error.	apps/web/src/app/admin/import/page.tsx apps/api/src/modules/products/bulkImport.ts
Downloadable import template	Completed	CSV and Excel templates.	apps/api/src/modules/products/routes.ts
Bulk import from Excel files	Completed	The .xlsx file is converted to CSV and goes through the same checks.	apps/api/src/modules/products/xlsxImport.ts

Feature	Status	Remarks	Main files
Barcode entry by scanner or typing	**Completed**	Keyboard-style scanners are detected. Not yet tested with a physical scanner.	`apps/web/src/hooks/useBarcodeScanner.ts` `apps/web/src/components/admin/ProductForm.tsx`
Searchable and filterable admin catalogue	**Partial**	Search by name, barcode or category and a category filter; no price-range or popularity filter.	`apps/web/src/app/admin/products/page.tsx`
Add or edit categories	**Not yet started**	The categories API is read-only and there is no screen.	`apps/api/src/modules/categories/routes.ts`
7. Inventory monitoring and stock adjustments			
Live stock dashboard with level indicators	**Completed**	Key figures, low-stock banner and stock levels.	`apps/web/src/app/admin/page.tsx`
Minimum stock level per item	**Completed**	For products and raw materials.	`apps/web/src/components/admin/ProductForm.tsx` `apps/web/src/components/admin/RawMaterialForm.tsx`
Low-stock list and banner	**Completed**	Dashboard, Inventory and Raw materials pages.	`apps/api/src/modules/inventory/routes.ts` `apps/web/src/app/admin/inventory/page.tsx`
Automated low-stock notifications by e-mail	**Completed**	Completed in code, not configured in production: a database trigger queues an alert and the worker e-mails it, but the production worker has no e-mail key.	`apps/api/prisma/migrations/20261013160000_low_stock_alerts/migration.sql` `apps/worker/src/jobs/sendLowStockAlerts.ts` `apps/worker/src/lib/mailer.ts`
Stock adjustments with a required reason	**Completed**	Damaged goods and spoilage can only lower stock.	`apps/web/src/components/admin/AdjustStockModal.tsx` `apps/web/src/components/admin/RawMaterialAdjustModal.tsx` `apps/api/src/modules/inventory/service.ts`
Adjustment log (who, when, before/after, reason)	**Completed**	Separate logs for products and raw materials.	`apps/web/src/app/admin/inventory/page.tsx` `apps/web/src/app/admin/raw-materials/page.tsx`
8. Analytics, reports and data export			
Daily, weekly and monthly sales summaries	**Completed**	Last 14 days, 8 weeks or 6 months.	`apps/api/src/modules/reports/service.ts` `apps/web/src/app/admin/analytics/page.tsx`
Top-selling products	**Completed**	All-time ranking.	`apps/api/src/modules/reports/service.ts`
Inventory movement	**Completed**	Units sold against current stock.	`apps/api/src/modules/reports/service.ts`
Profitability per item	**Completed**	Cost, price, margin and gross profit.	`apps/api/src/modules/reports/service.ts`
Revenue, transaction count and average order value	**Partial**	Revenue and order count shown; average order value not computed.	`apps/web/src/app/admin/analytics/page.tsx`

Feature	Status	Remarks	Main files
CSV export	**Completed**	Every report.	`apps/api/src/modules/reports/export.ts`
PDF and Excel export	**Completed**	Every report.	`apps/api/src/modules/reports/export.ts`
9. Authentication, roles and user management			
Staff and administrator sign-in	**Completed**	15-minute access token, 7-day refresh cookie, background renewal.	`apps/api/src/modules/auth/service.ts` `apps/api/src/lib/jwt.ts` `apps/web/src/app/login/page.tsx`
Anonymous kiosk session for customers	**Completed**	2-hour token, renewed automatically.	`apps/api/src/modules/auth/service.ts` `apps/web/src/lib/auth.ts`
Role-based access control (Administrator, Staff, Customer)	**Completed**	Checked on the server for every API route and on the pages.	`apps/api/src/middleware/authorize.ts` `apps/web/src/components/admin/AdminRouteGuard.tsx`
User administration	**Completed**	Create, change role, suspend or reactivate; an administrator cannot demote or suspend their own account.	`apps/web/src/app/admin/users/page.tsx` `apps/api/src/modules/users/service.ts`
Signed-out visitors kept out of staff pages	**Completed**	Redirect to sign-in; staff pages do not exist on the customer link.	`apps/web/src/components/StaffGuard.tsx` `apps/web/src/proxy.ts`
10. Security measures			
Encrypted transport (HTTPS)	**Completed**	All deployed services use HTTPS addresses on Railway.	—
SQL-injection prevention	**Completed**	Prisma parameterised queries; the only raw queries are in the backup restore check and use internal table names.	`apps/worker/src/jobs/restoreCheck.ts`
Password hashing and token security	**Completed**	bcrypt; refresh token in an httpOnly cookie.	`apps/api/src/modules/users/service.ts` `apps/api/src/modules/auth/routes.ts`
Input validation layer	**Completed**	Zod schemas on every module.	`apps/api/src/modules/orders/schema.ts` `apps/api/src/modules/products/schema.ts`
Security headers and CORS	**Completed**	helmet; only the kiosk and admin addresses may call the API.	`apps/api/src/app.ts` `apps/api/src/lib/origins.ts`
Ownership checks and signed payment webhooks	**Completed**	A kiosk session can read only its own orders; webhook signatures are verified.	`apps/api/src/modules/orders/routes.ts` `apps/api/src/lib/paymongo.ts`
Rate limiting	**Completed**	Sign-in, session renewal, order placement, kiosk sessions and payment start.	`apps/api/src/middleware/rateLimit.ts`

Feature	Status	Remarks	Main files
11. Background worker and backups			
Hourly low-stock check	**Completed**	Logs items at or below their minimum.	`apps/worker/src/jobs/lowStockSweep.ts`
Scheduled database backup	**Completed**	Nightly at 2:00 AM (Asia/Manila) with pg_dump.	`apps/worker/src/jobs/backupDatabase.ts` `apps/worker/src/index.ts`
Off-site backup copy	**Completed**	Uploaded to Cloudflare R2; a restore check job is provided.	`apps/worker/src/lib/storage.ts` `apps/worker/src/jobs/restoreCheck.ts`
Offline transaction queuing and synchronisation	**Removed from scope**	Removed from the manuscript by adviser decision.	—
12. Offline support (PWA)			
Installable Progressive Web App	**Removed from scope**	Removed from the manuscript by adviser decision.	—
Offline caching of the menu	**Removed from scope**	Removed from the manuscript by adviser decision.	—
Offline order queue with automatic synchronisation	**Removed from scope**	Removed from the manuscript by adviser decision.	—
13. Hardware integration			
Barcode scanner input	**Completed**	Implemented for keyboard-style USB or Bluetooth scanners; not yet tested with a physical scanner.	`apps/web/src/hooks/useBarcodeScanner.ts` `apps/web/src/app/admin/barcode/page.tsx`
Receipt printer	**Partial**	Kiosk side implemented; depends on the print service outside the repository (see module 2).	`apps/web/src/lib/receiptPrinter.ts`
14. QR code and multi-device access			
Kiosk usable on phones, tablets and desktops	**Completed**	Responsive layout; the back-office sidebar stays visible at every width.	`apps/web/src/app/page.tsx` `apps/web/src/components/admin/AdminShell.tsx`
QR code that opens the kiosk	**Completed**	Generated in the browser; printable poster and PNG download.	`apps/web/src/app/admin/kiosk-qr/page.tsx`
15. Deployment			
Web application and API on Railway	**Completed**	Separate customer (kiosk) and staff (admin) addresses.	`apps/web/src/proxy.ts`
Managed PostgreSQL on Railway	**Completed**	Migrations applied on API start.	`apps/api/prisma/schema.prisma`
Worker running in production	**Completed**	Builds from its own Dockerfile; cloud backups verified on 15 October 2026.	`apps/worker/Dockerfile`

Feature	Status	Remarks	Main files
16. Testing and quality assurance			
Automated unit tests	**Completed**	13 test files.	`apps/api/test/unit`
Automated integration and security tests	**Completed**	21 test files, including an access matrix for every API route.	`apps/api/test/integration`
Type-checking	**Completed**	No type errors in the API, web application or worker.	—
Linting	**Partial**	11 errors and 6 warnings reported by ESLint in the web application.	—
Documentation of test sessions	**Partial**	The testing document records 405 test cases; about 100 newer test cases are not yet documented.	—
Usability evaluation (ISO/IEC 25010)	**Not yet started**	Questionnaire prepared and approved by the adviser; not yet administered.	—
Structured testing on the physical kiosk	**Not yet started**	Planned. The Raspberry Pi kiosk and printer are set up and a test print was made, but no structured test has been run (ordering, printing, idle reset, scanner).	—

5. Completed Features

The completed work covers the full ordering cycle and the back-office tools around it. The figures are low-fidelity wireframes drawn from the current code (`docs/diagrams/`); each names the files it is based on.

5.1 Customer ordering

The kiosk opens on a welcome screen and asks once whether the customer will dine in or take out. The menu shows one category at a time, with a category rail, keyword search and price-range chips; sold-out dishes are dimmed and update live. Customers add dishes directly or through an item sheet with quantity, quick instruction options and a note. Checkout shows the order, an estimated preparation time based on the kitchen queue, and the payment choice (Pay at counter, or QR Ph once online payment is set up); placing the order returns a receipt with a unique order number. Inactivity triggers a warning and then resets the kiosk for the next customer.

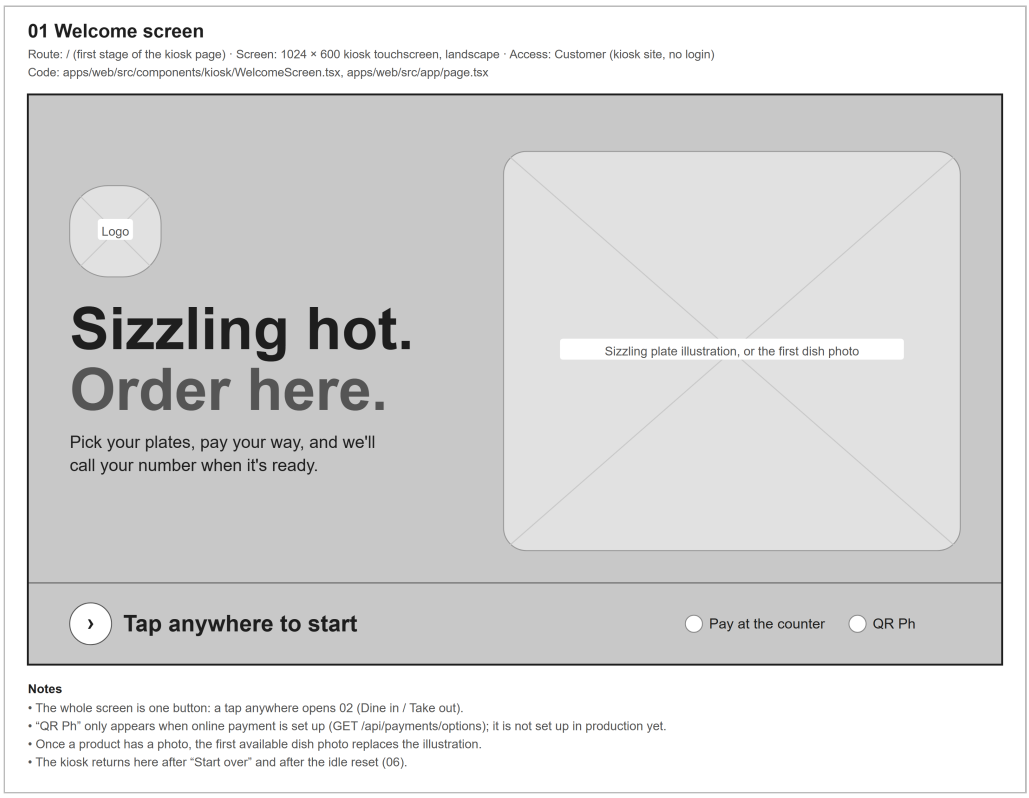


Figure 2. Kiosk welcome screen (wireframe).

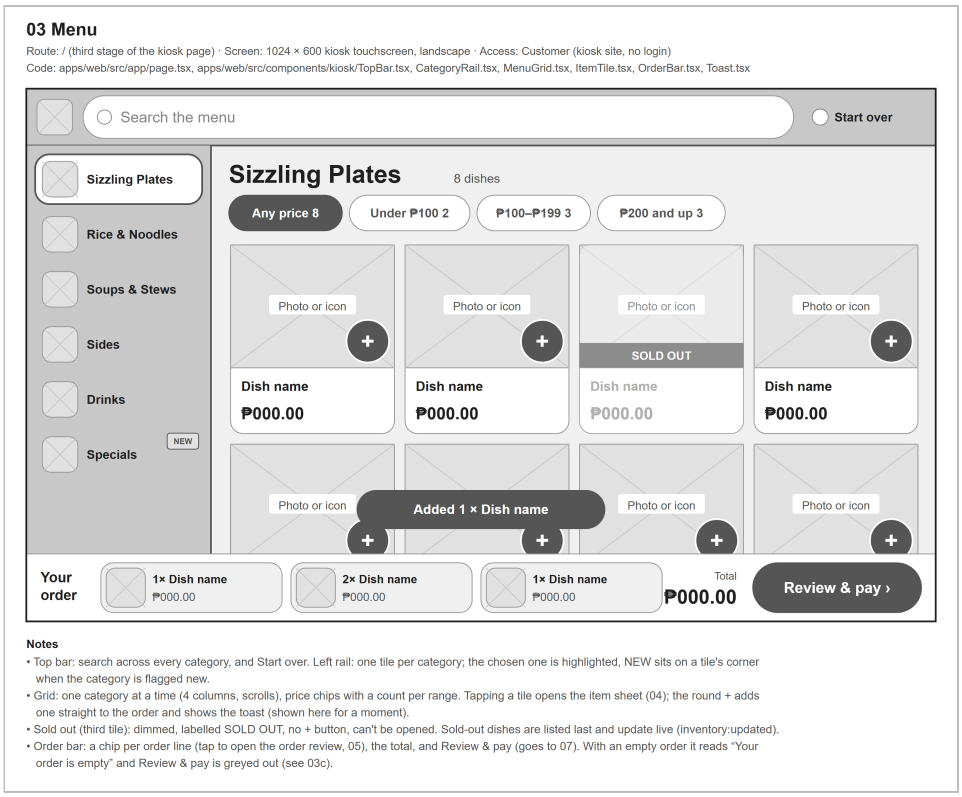


Figure 3. Kiosk menu with category rail, dish grid, price filter and order bar (wireframe).

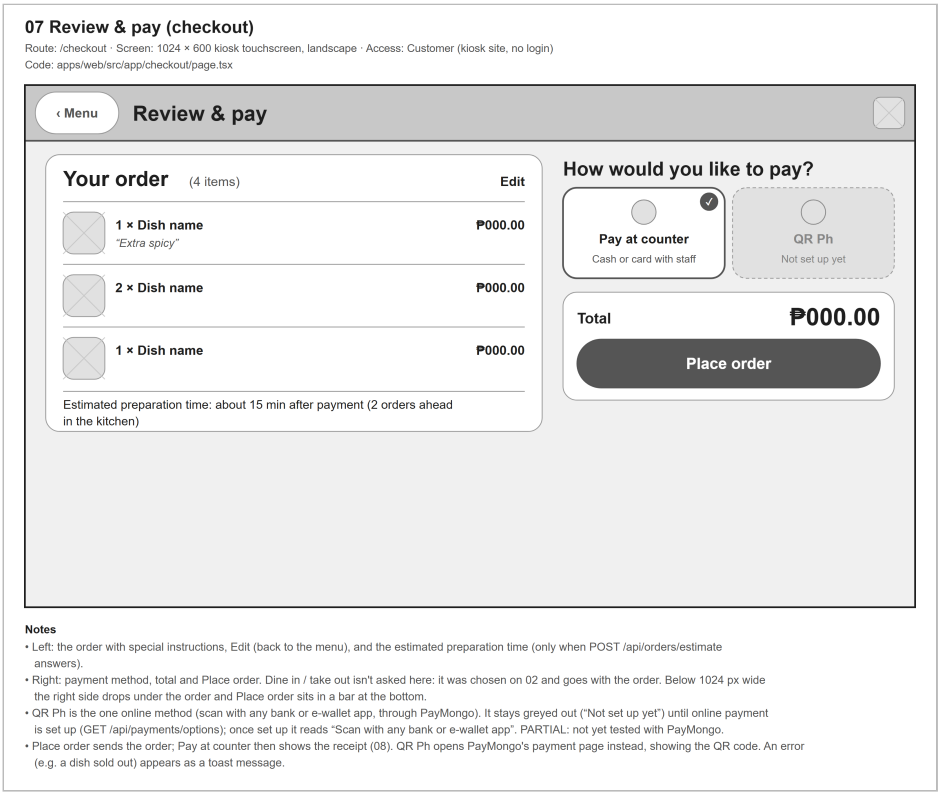


Figure 4. Review and pay screen (wireframe).

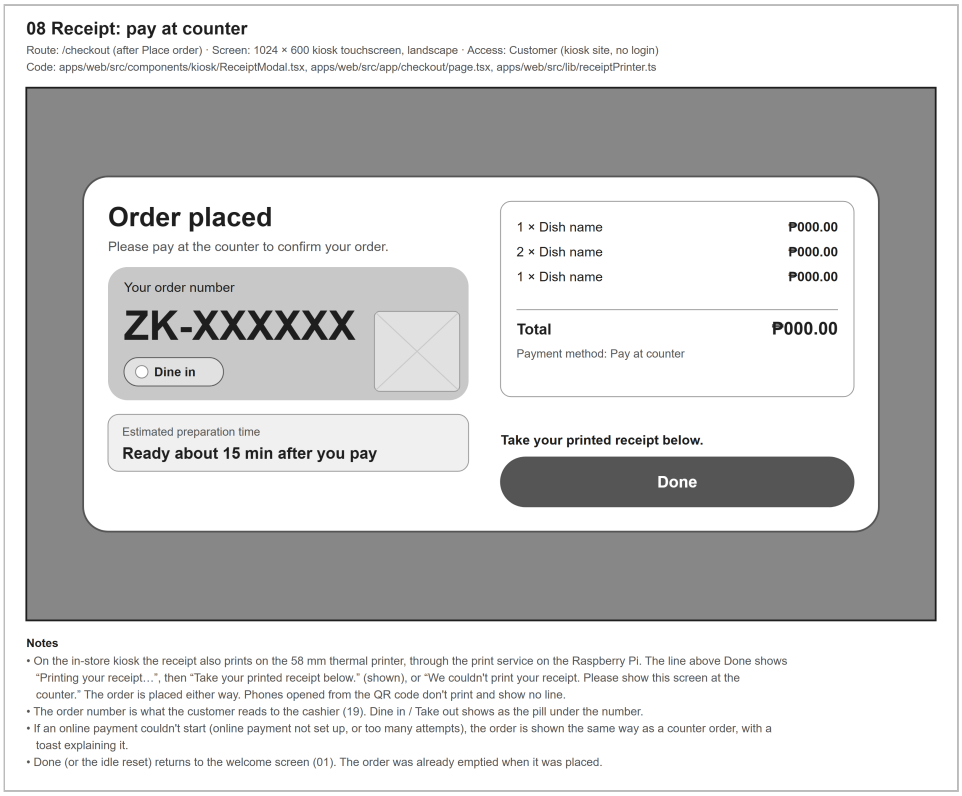


Figure 5. Receipt after a counter order, with the print status (wireframe).

5.2 Counter payment and kitchen

The cashier finds the order by its number in Orders awaiting payment and confirms it. Confirmation marks the order paid, deducts the stock and creates kitchen tasks in a single database transaction; if any item has run out, nothing changes and the cashier is told before money is taken. Unpaid orders can be cancelled. Paid orders appear immediately on the kitchen display, where each dish moves from Pending to In progress to Completed, with dine-in and take-out orders distinguished.

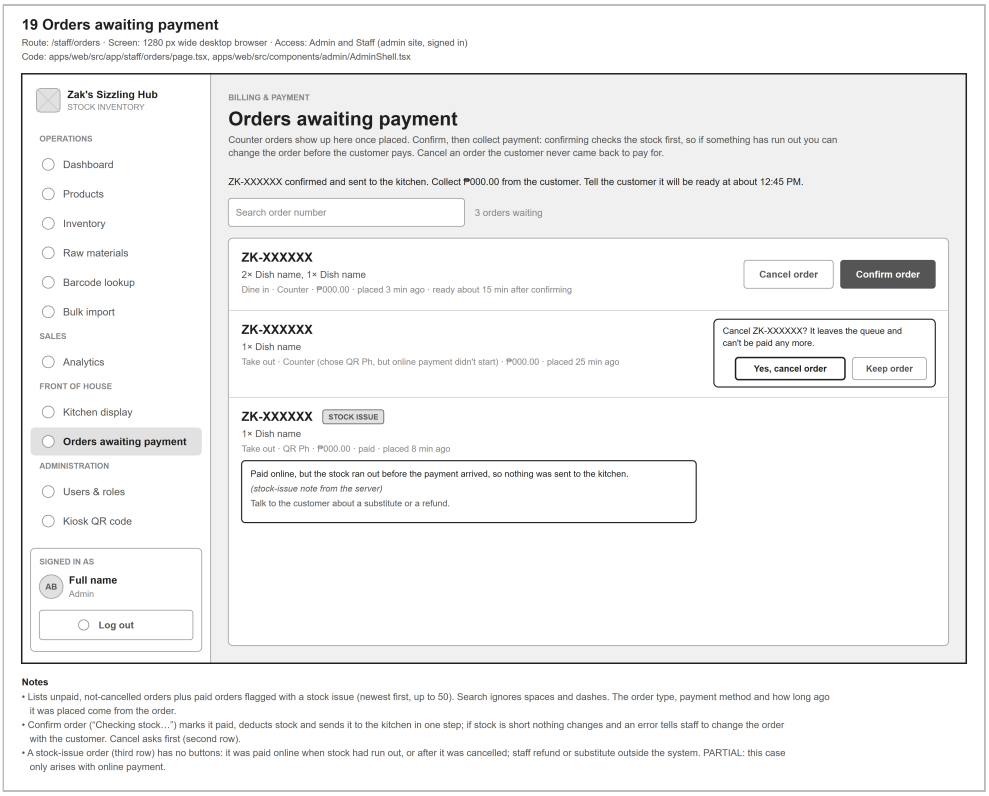


Figure 6. Orders awaiting payment, used by the cashier (wireframe).

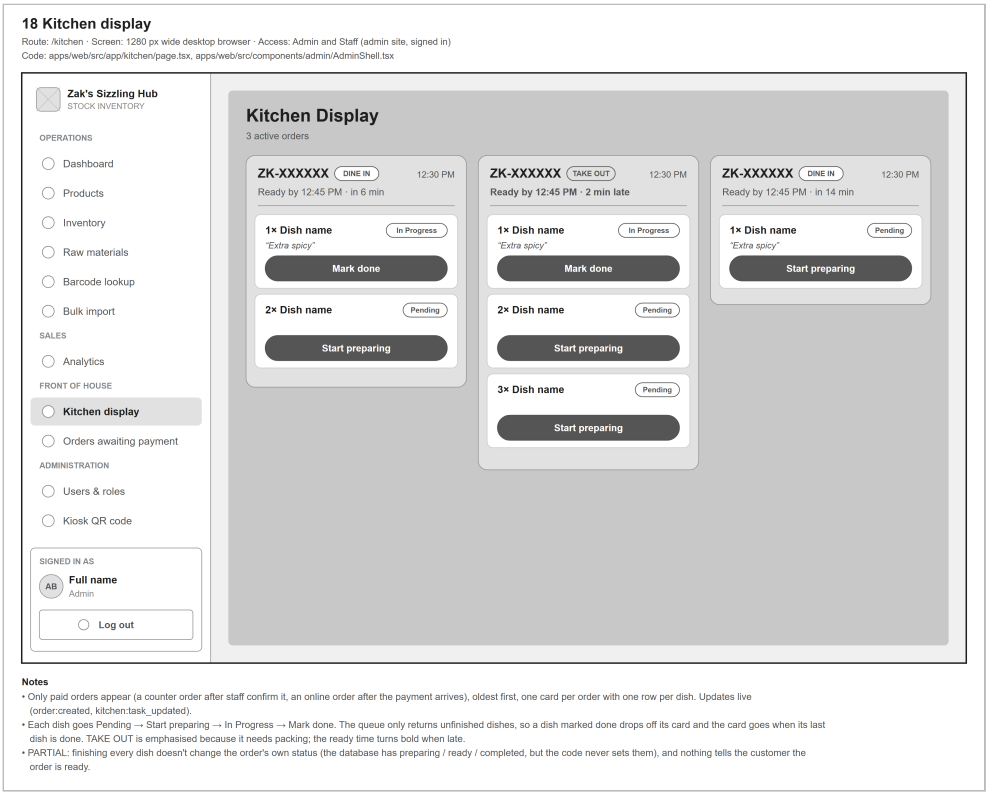


Figure 7. Kitchen display (wireframe).

5.3 Inventory

Stock is tracked per product (units) or per raw material for dishes made from a recipe; the kiosk shows how many servings the raw materials still allow. Each item has a minimum level, and items at or below it appear

on the dashboard and stock pages and are queued for an e-mail alert. Every manual change requires a reason and is logged with the person, the time and the before and after quantities. Products can also be found by scanning or typing their barcode.

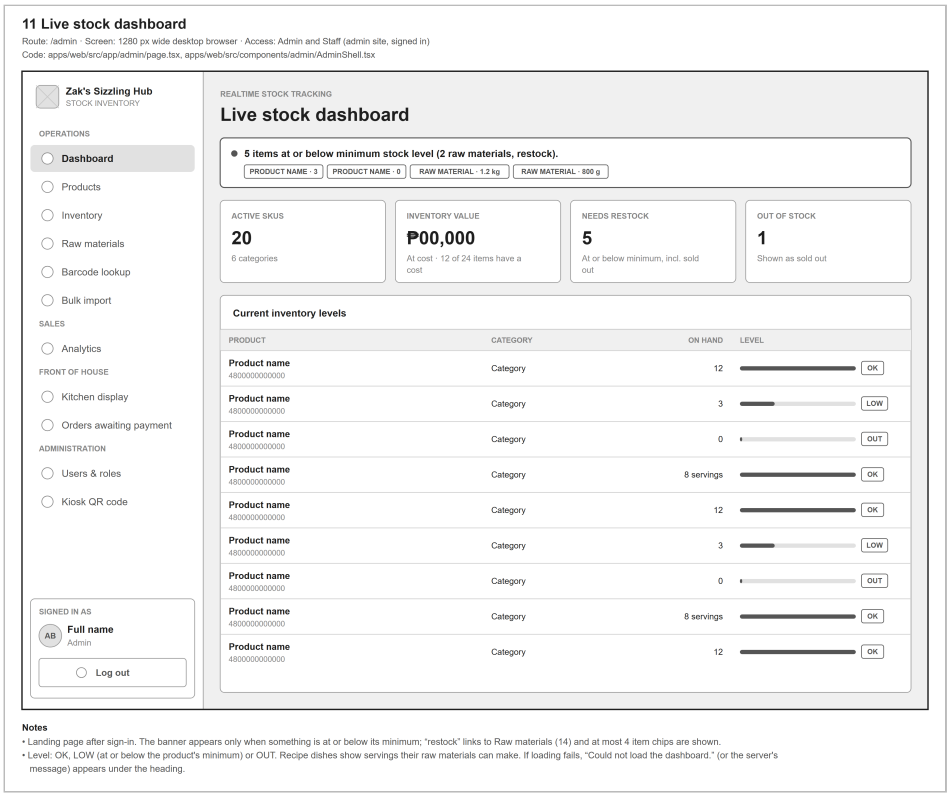


Figure 8. Live stock dashboard (wireframe).

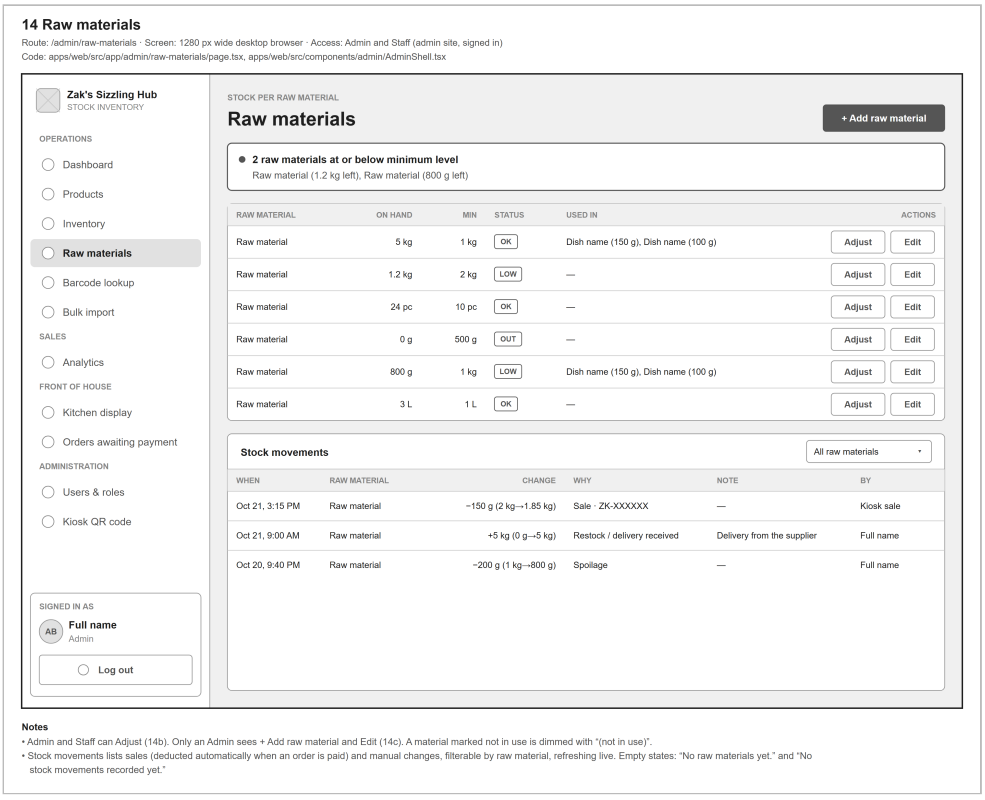


Figure 9. Raw-material stock and movement log (wireframe).

5.4 Products, analytics, users and operations

The Administrator creates and edits products with validation, photos, preparation times and recipes, and imports products in bulk from CSV or Excel after a row-by-row check. Analytics show daily, weekly and monthly sales, top-selling products, profitability and inventory movement, each exportable as CSV, Excel or PDF. User accounts can be created, assigned a role, suspended and reactivated. The Administrator also prints a QR code that opens the kiosk on customers' phones. In operation, the worker backs up the database every night to off-site storage, and separate web addresses keep staff pages off the customer link.

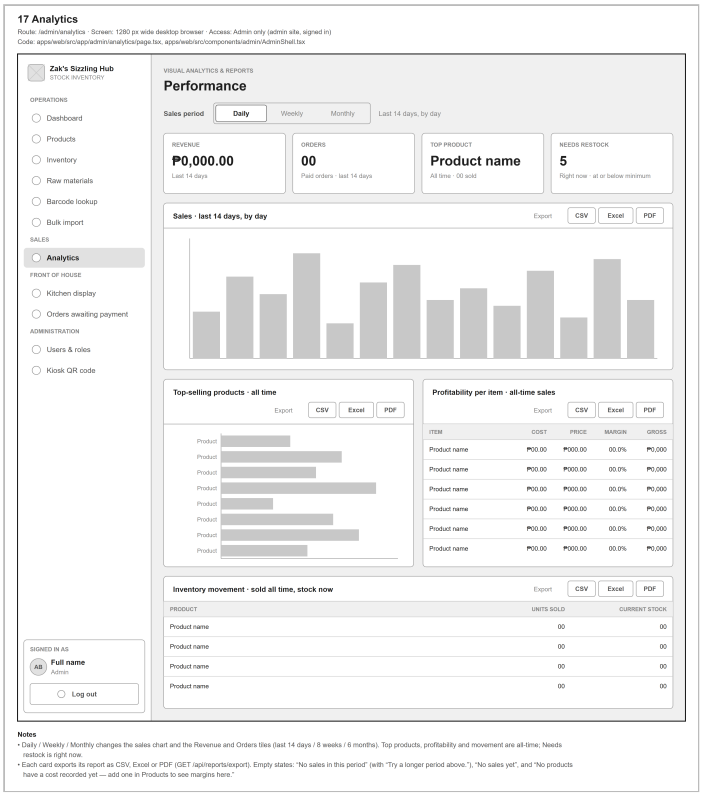


Figure 10. Analytics and report export (wireframe).

5.5 Security and quality

Passwords are hashed with bcrypt, staff sessions use short-lived tokens, all input is validated, responses carry security headers, only the kiosk and admin addresses may call the API, payment webhooks are verified by signature, and repeated sign-in, order and payment attempts are rate limited. The API and worker are covered by automated unit, integration and security tests (Section 8).

6. Partial and Pending Features

The table lists every feature that is Partial or Not yet started, with what exists and what remains.

Feature	Status	What is done and what is left
Item details: description, ingredients, allergens, nutrition 1. Kiosk ordering interface	Partial	Displayed on the kiosk. Ingredients, allergens and nutrition have no input screen; they can only be set in the database.
Filter by price range or popularity 1. Kiosk ordering interface	Partial	Price-range filter implemented; no popularity filter.
Online payment through PayMongo (QR Ph) 2. Checkout and payments	Partial	QR Ph is the one online method; it replaced GCash and Maya on 26 October 2026. Checkout session, redirect and signed webhook are implemented and covered by automated tests with a simulated gateway. Not yet run against PayMongo; production has no keys yet.
Receipt printing 2. Checkout and payments	Partial	After a counter order the kiosk sends the receipt to a print service on the Raspberry Pi (58 mm thermal printer). The print service is not in the repository and has not yet been verified with the deployed system.
QR-code payment option (manuscript wireframe) 2. Checkout and payments	Not yet started	Not present in the code.
Testing of PayMongo with sandbox and live keys 2. Checkout and payments	Not yet started	Planned. Requires PayMongo keys and the client's approval.
Transaction history screen 3. Front-desk order handling	Partial	The API lists recent orders (GET /api/orders); no screen displays it.
Order status progression and cancellation 3. Front-desk order handling	Partial	Cancelling an unpaid order is implemented. The statuses preparing, ready and completed are never set, and the customer is not notified when an order is ready.
Searchable and filterable admin catalogue 6. Product and catalogue management	Partial	Search by name, barcode or category and a category filter; no price-range or popularity filter.
Add or edit categories 6. Product and catalogue management	Not yet started	The categories API is read-only and there is no screen.
Revenue, transaction count and average order value 8. Analytics, reports and data export	Partial	Revenue and order count shown; average order value not computed.
Receipt printer 13. Hardware integration	Partial	Kiosk side implemented; depends on the print service outside the repository (see module 2).
Linting 16. Testing and quality assurance	Partial	11 errors and 6 warnings reported by ESLint in the web application.
Documentation of test sessions 16. Testing and quality assurance	Partial	The testing document records 405 test cases; about 100 newer test cases are not yet documented.

Feature	Status	What is done and what is left
Usability evaluation (ISO/IEC 25010) 16. Testing and quality assurance	Not yet started	Questionnaire prepared and approved by the adviser; not yet administered.
Structured testing on the physical kiosk 16. Testing and quality assurance	Not yet started	Planned. The Raspberry Pi kiosk and printer are set up and a test print was made, but no structured test has been run (ordering, printing, idle reset, scanner).

Removed from scope

Offline transaction queuing, the installable Progressive Web App, offline caching of the menu, and the offline order queue were removed from the manuscript by the adviser's decision (`docs/manuscript-fixes/CHANGES.md`, section 1). The system requires an internet connection.

7. Known Issues and Limitations

7.1 Functional

1. **Order status after cooking.** The database defines the statuses preparing, ready and completed, but the code never sets them; an order remains "confirmed" after the kitchen finishes, and the customer is not notified that it is ready (`apps/api/src/modules/kitchen/service.ts`).
2. **Changing an order.** When stock runs out at confirmation, the cashier is told to change the order with the customer, but no screen edits an existing order. The API provides routes to change or remove order items that the web application does not use; in practice the order is cancelled and placed again.
3. **No live refresh of the payment queue.** Orders awaiting payment loads when opened and after each action; a newly placed order appears only after the page is reloaded.
4. **Data without an input screen.** Categories, and each dish's ingredients, allergens and nutrition (shown on the kiosk), can only be changed directly in the database.
5. **Online payment gaps.** A customer who cancels on PayMongo's page returns to an empty checkout page, and a payment not confirmed within five checks is shown with the pay-at-counter wording. Neither case occurs in production while online payment is disabled.
6. **Receipt content.** The print service prints the order number, date, items and total, and always ends with "Please pay at the counter". It does not yet print the restaurant name or the order type, and receipts after an online payment are not printed.
7. **Account passwords.** Users cannot change their own password, and temporary passwords given to new users are not required to be changed.
8. **Low-stock e-mail in production.** The alerts are queued, but the production worker has no e-mail key, so no e-mail is sent there.

7.2 Security and maintenance

1. **Hosting provider incident.** On 21 October 2026 Railway disclosed unauthorised access to customer environment variables and advised rotating every secret. The three token secrets have been replaced; the database password and the backup-storage and e-mail keys remain to be rotated, and Railway paused deployments during its investigation.
2. **Dependency advisories.** `npm audit` reports 18 advisories in production dependencies (3 critical, 9 high, 6 moderate), all with fixes available. The critical advisory in Next.js concerns `next/og`, which the system does not use; the others should still be resolved by upgrading.
3. **Browser token storage.** Staff access tokens are kept in browser storage, which a cross-site scripting flaw could expose; the impact is limited by the 15-minute token lifetime and the httpOnly refresh cookie.
4. **Print service exposure.** The print service on the Raspberry Pi accepts requests from any web page, but listens only on the Pi itself.
5. **Lint findings.** ESLint reports 11 errors and 6 warnings in the web application (mostly state updates inside effects); type-checking reports none.
6. **No automated tests for the web interface.** The screens were checked manually in a browser.

8. Testing Status

Activity	Result	Details
Automated tests (API and worker)	**All passed**	Run on 26 October 2026: 514 test cases in 34 files; 510 passed, 4 skipped, 0 failed. The skipped cases are backup tests that require pg_dump on the test computer. No test is marked as a known defect.
Unit tests	13 files	Token handling, middleware, preparation-time formula, stock fulfilment, CSV and Excel import, report export, e-mail content, backup storage, allowed origins, the PayMongo checkout request.
Integration and security tests	21 files	Against a separate test database: an access matrix covering every API route and role, sign-in, orders, stock and raw materials, payments (simulated gateway), barcode lookup, rate limiting, malformed input, product images, imports, exports, low-stock alerts and backups.
Type-checking	No errors	API, web application and worker.
Linting	11 errors, 6 warnings	Web application (Section 7.2).
Test documentation	Partial	The testing document (14 October 2026) records 405 test cases over earlier sessions; about 100 newer test cases are not yet documented.
Web interface	Manual only	Checked in a browser at the kiosk size (1024 × 600) and on desktop; no automated tests.
Physical kiosk	Not yet started	Kiosk and printer set up and a test print made; no structured test yet (ordering, printing, idle reset, barcode scanner).
Online payment	Not yet started	QR Ph never run against PayMongo with sandbox or live keys.
Usability (ISO/IEC 25010)	Not yet started	Questionnaire prepared and approved by the adviser; not yet administered.

9. Next Steps

The remaining work is listed in the recommended order: security first, then the items the panel is most likely to ask to see, then evaluation, then functional gaps.

1. **Complete the secret rotation.** Replace the database password and the backup-storage and e-mail keys on Railway once deployments resume, then confirm that every service redeploys and that ordering, sign-in and backups still work.
2. **Verify receipt printing end to end.** Open the kiosk on the Raspberry Pi in printer mode, place counter orders and confirm that receipts print; add the restaurant name and order type to the receipt; place the print service's code in the repository.
3. **Conduct structured testing on the physical kiosk.** Ordering, printing, the idle reset, touch use at 1024 × 600, and the barcode scanner with a physical device; record the results.
4. **Test online payment (QR Ph).** With PayMongo sandbox keys first, then live keys after the client's approval; fix the cancelled-payment and delayed-confirmation cases.
5. **Administer the ISO/IEC 25010 evaluation** with the approved questionnaire and analyse the results.
6. **Document the newer test cases** in the testing document as a new session.
7. **Close the functional gaps:** order status progression with a ready notice, category management, input for ingredients, allergens and nutrition, average order value, a transaction history screen, live refresh of the payment queue, and password change for users.
8. **Maintenance:** upgrade dependencies with published advisories, resolve the lint errors, and configure the e-mail key so low-stock alerts are sent in production.